



**Leo Mora Granado, Alejandro Reoyo Giménez y Daniel
Azcárate Santafé**

Prácticas Presenciales 2ª Evaluación

Desarrollo de una aplicación web

Curso 2025-2026

1º DAM-DAW

**FUNDACIÓN SAN VALERO
SEAS, Centro de Formación Abierta
ZARAGOZA**

ÍNDICE

1. Propósito de la aplicación web	3
IntelliJ.....	3
Base de datos relacional	8
Flujo MVC (Modelo-Vista-Controlador)	9
Condiciones Obligatorias y Opcionales	11
2. Reflexión Tecnológica.....	14

1. Propósito de la aplicación web

Explicación clara de la aplicación.

Booksland es una aplicación web de gestión de libros y autores que permite visualizar, registrar, modificar y eliminar información almacenada en una base de datos relacional mediante una interfaz web responsive. Los pasos realizados han sido subidos al [GitHub](#), cuyo [README](#) explica toda la interfaz y funcionamiento de dicha tienda de libros de manera general.

A continuación, se explica todo el algoritmo Java incluyendo directorios, paquetes clases Java, y archivos de texto:

IntelliJ

- Booksland (Directory) 📁

/* Directorio raíz del proyecto que contiene toda la estructura de la aplicación web Java, incluyendo configuración de IntelliJ IDEA, código fuente, recursos, archivos SQL y configuración Maven. */

- .idea (Directory) 📁

/* Carpeta interna generada por IntelliJ IDEA donde se almacenan configuraciones del proyecto, módulos, control de versiones y preferencias del entorno de desarrollo. */

- .gitignore (File) 📄

/* Archivo que define qué elementos de configuración local del IDE no deben ser rastreados por Git. */

- Booksland.iml (File) 📄

/* Archivo de módulo de IntelliJ IDEA que almacena la configuración estructural y dependencias del módulo principal del proyecto. */

- misc.xml (File) 📄

/* Archivo de configuración general del proyecto dentro de IntelliJ, incluyendo versión del SDK y ajustes globales. */

- modules.xml (File) 📄

/* Define los módulos registrados en el proyecto IntelliJ y cómo se relacionan entre sí. */

- vcs.xml (File) 📄

/* Archivo encargado de configurar el sistema de control de versiones utilizado por el proyecto, como Git. */

- workspace.xml (File) 📄

/* Contiene configuraciones temporales y personales del entorno de trabajo del desarrollador, como pestañas abiertas y ejecución reciente. */

- Booksland (Directory) 📁

/* Carpeta principal que contiene el código fuente real de la aplicación web, recursos,

configuración de despliegue y archivos auxiliares del proyecto. */

- **.idea (Directory)** 
/* Configuración adicional del módulo interno del proyecto gestionado desde IntelliJ IDEA. */
- **src (Directory)** 
/* Directorio estándar que agrupa todo el código fuente y recursos utilizados por la aplicación. */
 - **main (Directory)** 
/* Carpeta destinada al contenido principal de la aplicación que será compilado y desplegado en el servidor web. */
 - **java (Directory)** 
/* Directorio que contiene todas las clases Java organizadas mediante paquetes siguiendo la estructura lógica de la aplicación. */
 - **com.svalero.booksland (Package)** 
/* Paquete base principal que agrupa todos los componentes Java de la aplicación Booksland. */
 - **dao (Package)** 
/* Paquete encargado de gestionar el acceso persistente a los datos de la base de datos mediante operaciones CRUD. */
 - **AuthorDAO.java (Java Class)** ©
/* Clase DAO responsable de realizar operaciones SQL relacionadas con los autores, como inserciones, consultas, modificaciones y eliminaciones. */
 - **BookDAO.java (Java Class)** ©
/* Clase DAO encargada de gestionar todas las operaciones persistentes asociadas a los libros almacenados en la base de datos. */
 - **Database.java (Java Class)** ©
/* Clase centralizada que administra la conexión JDBC con la base de datos, proporcionando métodos para abrir conexiones y reutilizar la configuración de acceso. */
 - **mapper (Package)** 
/* Paquete que contiene clases encargadas de transformar los resultados SQL (ResultSet) en objetos Java del modelo. */
 - **AuthorMapper.java (Java Class)** ©
/* Clase especializada en convertir registros obtenidos de la tabla de autores en objetos Author. */
 - **BookMapper.java (Java Class)** ©
/* Clase responsable de mapear los datos de

la tabla de libros hacia objetos Book dentro de la aplicación. */

▪ **model (Package)**

/* Paquete que contiene las entidades o modelos POJO que representan la estructura de datos principal del sistema. */

- **Author.java (Java Class)** ©

/* Modelo que representa la entidad Autor, incluyendo atributos como nombre, nacionalidad u otra información asociada al autor. */

- **Book.java (Java Class)** ©

/* Clase modelo que define la estructura de un libro dentro del sistema, incluyendo propiedades como título, género, ISBN o autor relacionado. */

▪ **servlet (Package)**

/* Paquete encargado de gestionar las peticiones HTTP recibidas desde el cliente, actuando como controlador entre la interfaz web, la lógica de negocio y el acceso a datos. */

- **DeleteAuthor.java (Java Servlet)**

/* Servlet responsable de procesar la eliminación de autores desde la aplicación web, ejecutando operaciones de borrado sobre la base de datos. */

- **DeleteBook.java (Java Servlet)**

/* Servlet encargado de eliminar registros de libros existentes en la base de datos a partir de solicitudes realizadas desde la interfaz web. */

- **EditAuthor.java (Java Servlet)**

/* Servlet utilizado para modificar la información asociada a un autor, gestionando tanto la carga de datos como la actualización persistente en la base de datos. */

- **EditBook.java (Java Servlet)**

/* Servlet responsable de gestionar la edición y actualización de la información de libros almacenados en el sistema. */

- **ListAuthor.java (Java Servlet)**

/* Servlet encargado de obtener y mostrar el listado completo de autores registrados en la base de datos mediante vistas web dinámicas. */

- **ListBook.java (Java Servlet)** 
/* Servlet que recupera y presenta el catálogo de libros almacenados en la aplicación para su visualización en el frontend. */
- **ViewAuthor.java (Java Servlet)** 
/* Servlet responsable de mostrar la información detallada de un autor específico seleccionado por el usuario desde el listado principal. */
- **ViewBooks.java (Java Servlet)** 
/* Servlet encargado de visualizar los detalles completos de un libro concreto, incluyendo su información relacionada y datos asociados. */
- **resources (Directory)** 
/* Carpeta destinada a almacenar recursos auxiliares de la aplicación, como archivos de configuración y propiedades. */
 - **db.properties (File)** 
/* Archivo de configuración que almacena los parámetros de conexión a la base de datos, como URL, usuario y contraseña JDBC. */
- **webapp (Directory)** 
/* Directorio principal de la aplicación web Java EE donde se encuentran los recursos de despliegue y configuración del servidor. */
 - **includes (Directory)** 
/* Carpeta que contiene fragmentos reutilizables de vistas JSP utilizados para mantener una estructura modular y evitar duplicación de código HTML. */
 - **footer.jsp (JSP File)** 
/* Archivo JSP reutilizable que define el pie de página común de la aplicación web, incluyendo información visual y estructura compartida. */
 - **header.jsp (JSP File)** 
/* Archivo JSP reutilizable que contiene la cabecera principal de la aplicación web, incluyendo navegación, estilos y elementos comunes de interfaz. */
 - **META-INF (Directory)** 
/* Carpeta que contiene archivos de configuración especiales utilizados por el contenedor web durante el despliegue de la aplicación. */
 - **context.xml (XML File)**  `</>`
/* Archivo de configuración del contexto de la aplicación en el servidor, utilizado normalmente

para definir recursos JNDI o parámetros del entorno web. */

- **WEB-INF (Directory)**

/* Carpeta protegida de la aplicación web que almacena configuraciones internas no accesibles directamente desde el navegador. */

- **web.xml (XML File)**

/* Archivo descriptor de despliegue de la aplicación web Java EE, donde se configuran servlets, filtros, listeners y rutas principales de la aplicación. */

- **authors.jsp (JSP File)**

/* Archivo JSP que renderiza la vista con el listado o gestión de los autores de la aplicación. */

- **books.jsp (JSP File)**

/* Archivo JSP encargado de mostrar la interfaz visual para el catálogo o listado general de libros. */

- **edit-author.jsp (JSP File)**

/* Vista JSP destinada a mostrar el formulario de edición de autores, permitiendo modificar y actualizar la información existente en la base de datos. */

- **edit-book.jsp (JSP File)**

/* Página JSP encargada de gestionar la modificación de datos asociados a los libros mediante formularios interactivos conectados al backend. */

- **index.jsp (JSP File)**

/* Página de inicio o vista principal de la aplicación web. */

- **view-author.jsp (JSP File)**

/* Archivo JSP diseñado para mostrar la interfaz con la información detallada de un autor específico. */

- **view-book.jsp (JSP File)**

/* Archivo JSP que genera la vista detallada de un libro concreto para el usuario. */

- **.gitignore (File)**

/* Archivo que indica a Git qué archivos temporales o generados automáticamente no deben incluirse en el repositorio. */

- **pom.xml (File)**

/* Archivo principal de configuración de Maven que define dependencias, plugins, versión de Java y proceso de construcción del proyecto. */

- **modelo-relacional.lua (File)**

/* Archivo auxiliar que contiene la representación del modelo relacional de la base de datos utilizando sintaxis Lua. */

- **modelo_entidad-relacion.graphml (File)**

/* Archivo que almacena el diagrama entidad-relación del sistema en formato GraphML para su visualización y edición gráfica. */

- Script-Creacion_Tablas.sql (SQL Script) 

/* Script SQL encargado de crear las tablas y estructuras necesarias de la base de datos de la aplicación Booksland. */

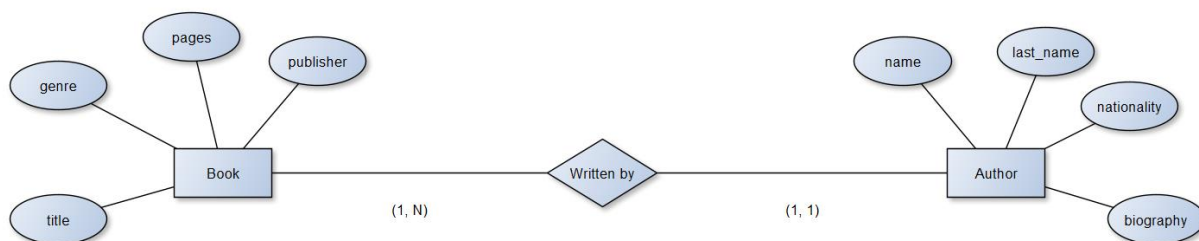
External Libraries

/* Referencia a todas las librerías y dependencias externas necesarias para el funcionamiento del proyecto, incluyendo JDK, JDBC y librerías gestionadas por Maven. */

Scratches and Consoles (Scratch File)

/* Espacio temporal proporcionado por IntelliJ IDEA para realizar pruebas rápidas de código, consultas SQL o experimentos sin modificar el proyecto principal. */

Base de datos relacional



1. **Book:** Catálogo de artículos de la tienda.

1.1. Id 

1.2. Title 

1.3. Genre 

1.4. Pages 

1.5. Publisher 

2. **Author:** Escritor del libro.

2.1. Id 

2.2. Name 

2.3. Last name 

2.4. Nationality 

2.5. Biography 

3. **Book_author:** Tabla intermedia de relación.

3.1. Id book 

3.2. Id author 

Script de Creación SQL

```
CREATE DATABASE IF NOT EXISTS booksland;
USE booksland;
```

```
CREATE TABLE books (
```

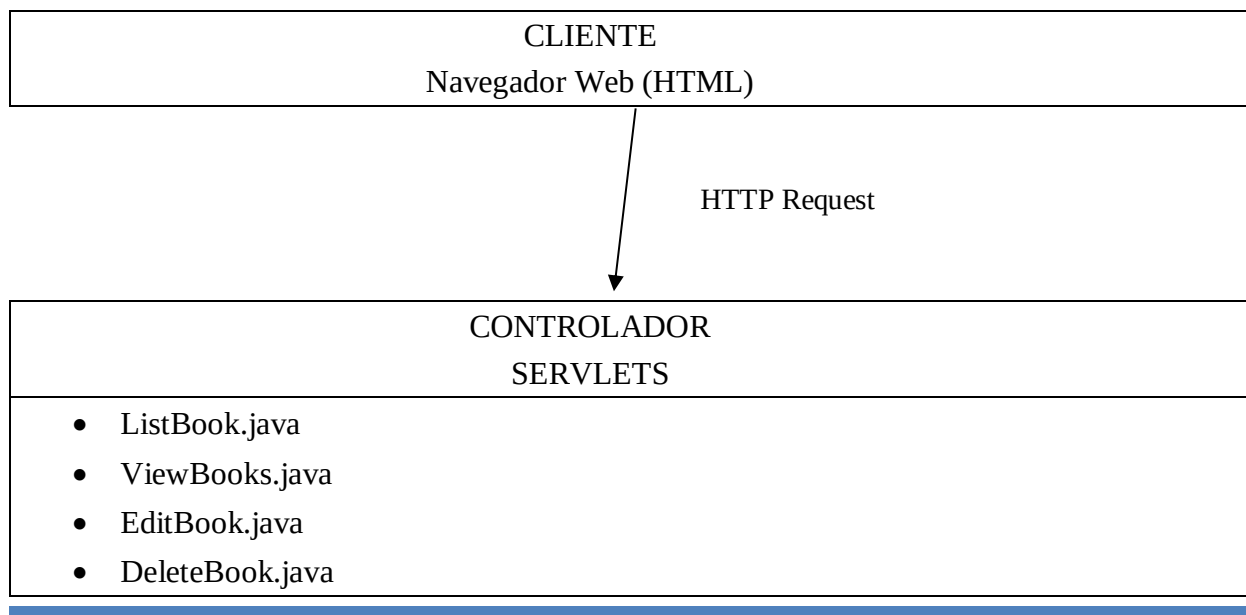
```

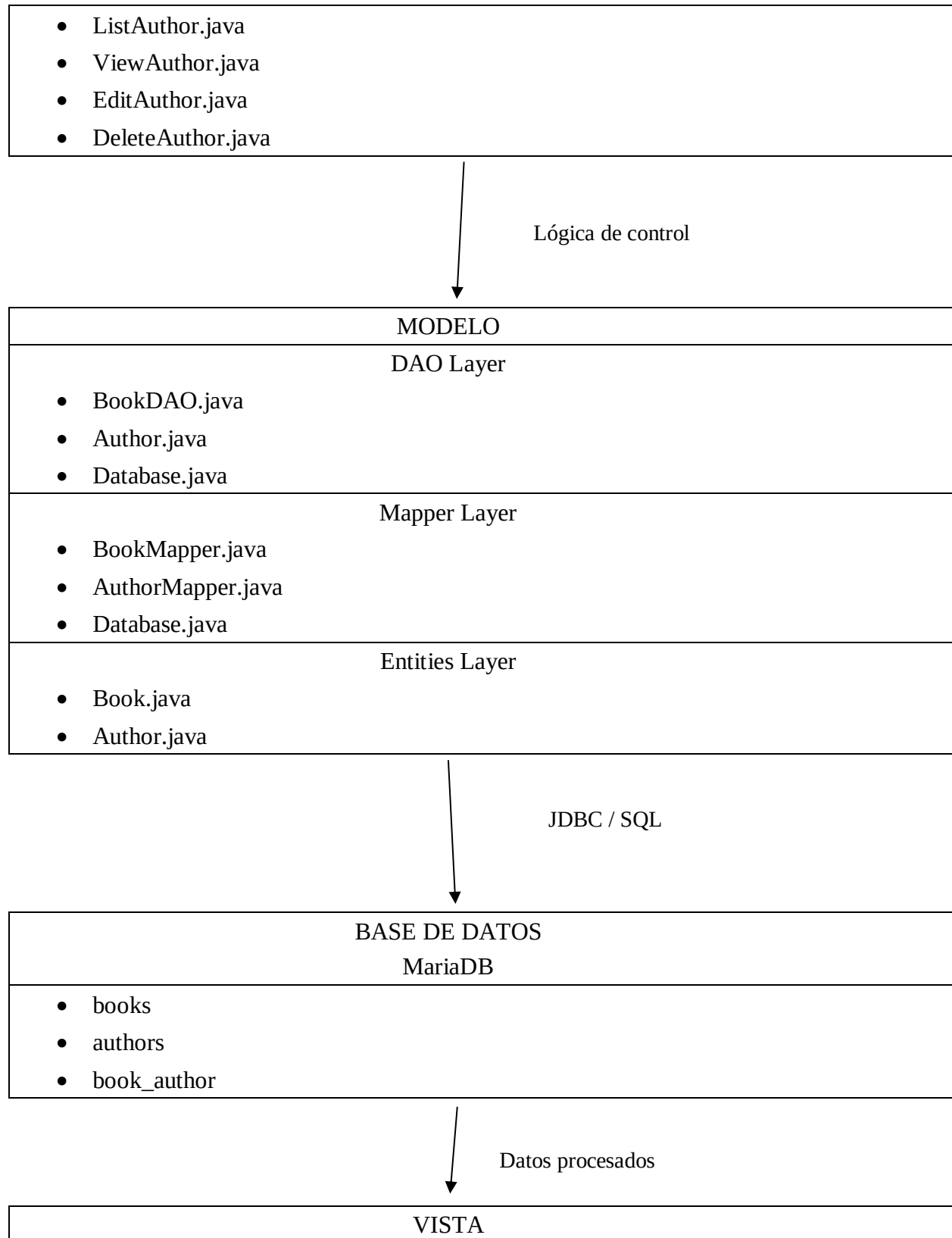
        id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
        title VARCHAR(50) NOT NULL,
        genre VARCHAR(50) NOT NULL,
        pages INT NOT NULL,
        publisher VARCHAR(50) NOT NULL
    );

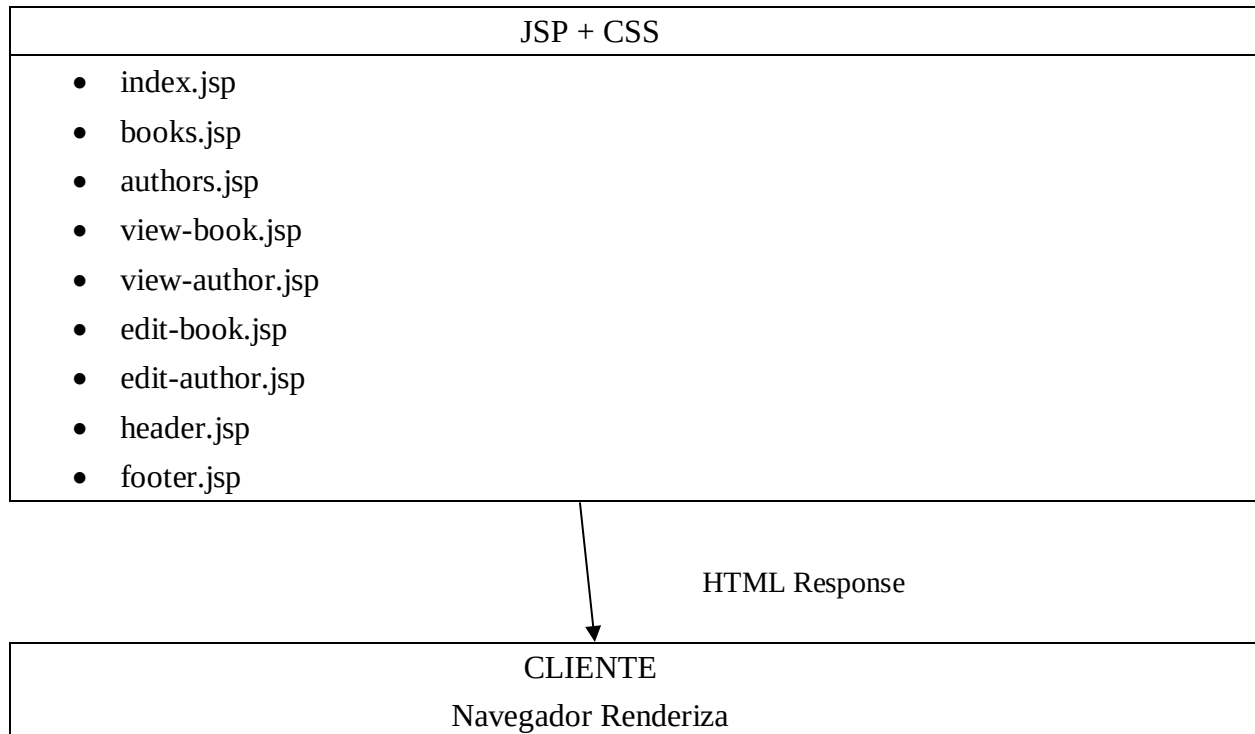
    CREATE TABLE authors (
        id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
        name VARCHAR(100) NOT NULL,
        last_name VARCHAR(100) NOT NULL,
        nationality VARCHAR(50),
        biography VARCHAR(250)
    );

    CREATE TABLE book_author (
        id_book INT UNSIGNED NOT NULL,
        id_author INT UNSIGNED NOT NULL,
        PRIMARY KEY (id_book, id_author),
        FOREIGN KEY (id_book) REFERENCES books(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
        FOREIGN KEY (id_author) REFERENCES authors(id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
    );
    
```

Flujo MVC (Modelo-Vista-Controlador)







Explicación resumida del flujo MVC

Modelo	Vista (View)	Controlador (Controller)
Gestiona los datos y la lógica de acceso a base de datos: - DAO → consultas SQL CRUD. - Mapper → convierte ResultSet en objetos Java. - Model → entidades Book y Author.	Formada por archivos JSP: - Muestran información. - Renderizan HTML dinámico. - Utilizan Bootstrap/CSS.	Los Servlets: - Reciben peticiones HTTP. - Llaman a los DAO, - envían datos a las vistas JSP.

Condiciones Obligatorias y Opcionales

Requisitos (1 pto cada uno, obligatorios)

- **Base de datos:** Diseña el modelo Entidad-Relación, prepara el modelo relacional y escribe el script SQL que permitirá crear la estructura de tablas que formará la base de datos del proyecto.
- **Funcionalidad (Listado y detalle):** La aplicación web debe listar el contenido de un par de tablas y visualizar los detalles de cada registro cuando se haga click sobre ellos. Se deberá maquetar utilizando HTML + CSS para presentar correctamente el contenido.
- **Funcionalidad (Registro y Borrado):** Se podrá registrar y eliminar información de un par de tablas (maquetando también con HTML + CSS) a través de un formulario.
- **Inglés:** Traduce todos los textos de la aplicación web para que ésta se muestre completamente en inglés.
- **Git Flow:** El código de la aplicación estará subido en un repositorio de **GitHub**. Se añadirá un fichero README con la información del proyecto así como las instrucciones para su puesta en marcha (backend + frontend). La versión final que se entregue se etiquetará como 1.0 y se incluirá como una Release en dicho repositorio. Cada nueva funcionalidad o incidencia se habrá gestionado como una nueva rama de trabajo que se habrá fusionado como **Pull Requests** tras ser revisada por otro compañero del equipo. Se añadirán al menos 4 tags para etiquetar 4 estados intermedios del proyecto.

Otras funcionalidades (1 pto cada una)

Completos:

- **Funcionalidad (modificar):** Se podrá modificar la información de cualquier elemento desde su vista detalle.
- **Funcionalidad (búsqueda):** Modifica la web para que los usuarios puedan realizar algún tipo de búsqueda o filtrado sobre los listados de datos.
- **Funcionalidad (relaciones):** Implementa funcionalidades en la web que permitan establecer relaciones entre datos de diferentes tablas.
- **Funcionalidad (Javascript):** Utiliza fetch para ejecutar de forma asíncrona alguna acción como registrar, eliminar o modificar algún elemento de la misma.
- **Diseño:** Utiliza bootstrap para realizar la maquetación de la web.
- **Diseño responsive:** Realiza una maquetación responsive de la web de forma que esta sea capaz de adaptarse tanto a dispositivos móviles como a desktop.

Incompletos:

- **Funcionalidad (Paginación):** A la hora de mostrar los listados, implementa alguna forma de paginación cuando estos listados superen un límite establecido.
- **Despliegue:** Se desplegará la aplicación web en AWS

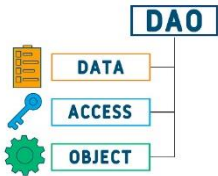
2. Reflexión Tecnológica

Herramientas utilizadas y sus funcionalidades.

SECCIÓN	TECNOLOGÍA	FUNCIONALIDAD
BACKEND/SERVIDOR	Java 	Lenguaje de programación principal del lado del servidor encargado de implementar la lógica de negocio robusta, gestionar la concurrencia y procesar las peticiones del backend.
	Jakarta Servlet API  JAKARTA EE	Interfaz de programación que actúa como capa intermedia para interceptar, procesar y responder de forma dinámica a las peticiones HTTP enviadas por los clientes web.
	Apache Tomcat 	Servidor de aplicaciones web y contenedor de servlets encargado de desplegar el código del backend, gestionar el ciclo de vida de la aplicación y coordinar las conexiones de red.
	Maven	Herramienta de automatización de compilación y gestión de dependencias encargada

		de unificar el ciclo de vida del software, descargar librerías externas y empaquetar el proyecto.
FRONTEND	HTML 	Lenguaje de marcado utilizado para estructurar y organizar el contenido visual de la aplicación web, definiendo elementos como formularios, tablas, enlaces, textos e imágenes que serán interpretados y mostrados por el navegador.
	CSS 	Lenguaje de estilos empleado para diseñar y personalizar la apariencia visual de la aplicación web, permitiendo definir colores, tipografías, espaciados, distribución de elementos y comportamiento responsive de la interfaz de usuario.
	Bootstrap 	Framework de diseño de interfaz de usuario empleado para estructurar un frontend responsive, ágil y visualmente coherente mediante componentes CSS y JavaScript preestablecidos.
	JavaScript	Lenguaje de programación interpretado del lado del

	JavaScript 	cliente encargado de dotar de interactividad dinamicidad a la interfaz de usuario, manipular el DOM en tiempo real, validar formularios y gestionar peticiones asíncronas con el backend.
	<u>AJAX</u> 	Técnica de desarrollo web empleada para actualizar partes de una página web de forma asíncrona, permitiendo intercambiar datos con el servidor y modificar el contenido sin necesidad de recargar la página completa.
	<u>JSP</u> 	Tecnología de vista en el servidor encargada de generar interfaces web dinámicas mediante la inserción de lógica Java dentro de plantillas HTML y su posterior compilación automática en servlets.
BASE DE DATOS	<u>MARIADB / MARIADB Java Client (JDBC Driver)</u> 	MARIADB: Sistema de gestión de bases de datos relacionales encargado del almacenamiento físico estructurado, la persistencia a largo plazo y la integridad de toda la información del sistema. MARIA DB Java Client

		(JDBC Driver): Componente de conectividad de bajo nivel que habilita la comunicación directa, el envío de consultas y la recepción de datos entre la aplicación Java y el servidor MariaDB.
ACCESO A DATOS	JDBI 3 	Librería de abstracción SQL de alto nivel integrada para simplificar la ejecución de consultas, automatizar el mapeo de registros relacionales a objetos Java y gestionar de forma limpia los recursos de conexión.
	Patrón DAO 	Patrón de diseño de software implementado para aislar por completo la lógica de negocio de los detalles técnicos del acceso a la base de datos, garantizando modularidad y mantenibilidad.
OTRAS LIBRERÍAS	Lombok 	Librería de automatización de código encargada de eliminar el código repetitivo (boilerplate) en Java mediante anotaciones, generando automáticamente en tiempo de compilación los métodos getters, setters, constructores y métodos toString.